

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory Work No. 4

Cryptography

Block Ciphers and DES

Computing the right half of a DES round

Task 2.8 – Group list number 19

Supervisors:
asist. univ. **Zaica Maia**
conf. univ., dr. **Zgureanu Aureliu**
DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Task and short theory **3**

1.1 Assigned task 3

1.2 Where the calculation fits in DES 3

1.3 The DES permutation P 3

2 Implementation **4**

2.1 Console input 4

2.2 Bit selection and XOR 4

2.3 Project entry point 4

3 Program output **5**

4 Conclusions **6**

Bibliography **6**

1 Task and short theory

1.1 Assigned task

My number in the group list is $n = 19$. The assignment uses n mod 11 to select the exercise, so $19 \bmod 11 = 8$ gives Task 2.8. The task is to compute the 32-bit right half R_k of a DES round from the previous left half L_{k-1} and the combined output of the eight S-boxes. The application must accept manual or randomly generated inputs and display the table and intermediate results used in the calculation [1].

1.2 Where the calculation fits in DES

DES processes 64-bit blocks in 16 Feistel rounds. Its 64-bit key representation includes eight parity bits, leaving 56 bits for the key schedule. An initial permutation produces the two 32-bit halves L_0 and R_0. Each round uses a 48-bit subkey K_k and computes

$$L_k = R_{k-1}, \qquad R_k = L_{k-1} \oplus f(R_{k-1}, K_k).$$

After the last round, the inverse initial permutation is applied to $R_{16} \| L_{16}$ to obtain the ciphertext block [2].

Inside f, expansion E maps R_{k-1} to 48 bits, which are XORed with K_k. The result is split into eight 6-bit blocks $B_1, \ldots, B_8$. Each S-box uses the outer two bits of its input to select a row and the inner four bits to select a column, producing four output bits. Concatenating the eight outputs yields the 32-bit word $S = S_1(B_1) \| \cdots \| S_8(B_8)$. Task 2.8 starts with this word already supplied, so the program computes $f = P(S)$ and then $R_k = L_{k-1} \oplus P(S)$.

1.3 The DES permutation P

Read Table 1.1 left to right, row by row. Its j-th entry is the input position copied to output position j. Positions are counted from the most significant bit, starting at one. Thus the first output bit is input bit 16, and the last is input bit 25 [2].

Table 1.1 - Input bit positions in output order

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

2 Implementation

2.1 Console input

The application is written in C# and targets .NET 10. Option `1` prompts for L_{k-1} and S in hexadecimal. `Convert.ToUInt32(input, 16)` accepts both `0x12345678` and `12345678`. An invalid menu choice repeats the menu prompt. A format error or a value larger than `0xFFFFFFFF` repeats the corresponding hexadecimal prompt.

Option `2` uses `Random.Next(0, int.MaxValue)` to generate each input. This limits generated words to `0x00000000` through `0x7FFFFFFE`, so their most significant bit is always zero. Manual input accepts the full unsigned 32-bit range.

2.2 Bit selection and XOR

`Program.cs` passes S and L_{k-1} to `Algorithm`. The method `P` stores the permutation as a 32-entry byte array and prints it before selecting bits. To read input position p, the code shifts the 32-bit unsigned word right by $32 - p$ and masks it with `1`. It shifts the partial result left and appends the selected bit using bitwise OR. After 32 iterations the accumulator contains $P(S)$.

```
uint result = 0;
for (byte i = 0; i < 32; i++)
{
    byte i_th_bit_index = (byte)(P_table[i] - 1);
    byte i_th_bit_value = (byte)((input >> (32 - i_th_bit_index - 1)) & 1);

    result = (result << 1) | i_th_bit_value;
}

return result;
```

The constructor computes `this.Rk = this.Ln_minus_one ^ permutated`. The C# `^` operator XORs corresponding bits, yielding one where they differ. The trace shows the inputs, the P table, the permuted word, and the XOR operands and result. Binary words are padded to 32 bits and hexadecimal words to eight digits. The main program prints the value returned by `getResult()`.

2.3 Project entry point

Run the following command from the repository root.

```
dotnet run --project lab_04/code/DES_cipher.csproj
```

The console entry point runs the two steps assigned in Task 2.8.

The Task 2.8 entry point takes the S-box output directly and uses `task_08/Algorithm.cs` for the round calculation.

3 Program output

This run uses option 2 to generate both input words.

```
User@Mac Cryptography_UTM %  dotnet run --project lab_04/code/DES_cipher.csproj
Hello. You are at the start of the 8-th task program. I need to ask you, would you like to introduce data
    or to generate it?
Enter '1' to introduce data or '2' to generate it randomly.
2
You have chosen to generate data randomly.
Randomly generated L[k-1] = 0x6356DBD6 and S-boxes result = 0x6D3D8BCA
Data generated
=== Part of the algorithm started. ===
You have provided the following inputs:
L[k-1] = 0x6356DBD6
S-boxes result = 0x6D3D8BCA

The same in binary format:
L[k-1] = 01100011010101101101101111010110
S-boxes result = 01101101001111011000101111001010

Step 1: Apply permutation P to the S-boxes result to obtain f.
Permutation P table:
16, 7, 20, 21, 29, 12, 28, 17, 1, 15, 23, 26, 5, 18, 31, 10, 2, 8, 24, 14, 32, 27, 3, 9, 19, 13, 30, 6,
    22, 11, 4, 25
Input to permutation P: 0x6D3D8BCA (01101101001111011000101111001010)
Result of permutation P: f = 0x9D3AF255 (10011101001110101111001001010101)

Step 2: Calculate R[k] by performing XOR between L[k-1] and f.
R[k] = L[k-1] XOR f = 0x6356DBD6 XOR 0x9D3AF255 = 0xFE6C2983
The XORing in binary:
01100011010101101101101111010110 XOR 10011101001110101111001001010101 = 11111110011011000010100110000011

[Result] R[k] = 0xFE6C2983
=== Part of the algorithm finished. ===
User@Mac Cryptography_UTM %
```

4 Conclusions

In this laboratory work, I implemented the last two steps of the DES round function and update needed to find the new right half. The assignment starts with the combined output of the eight S-boxes, so the same calculation can be used for any round once that word and the previous left half are available.

The permutation required careful attention to bit numbering. The DES table counts positions from the most significant bit, while the C# code uses right shifts to read a bit. Translating a table position p into a shift of $32 - p$ lets the program build the output word in the order specified by the table.

The XOR then combines the permuted word with the previous left half. Its reversibility is part of the Feistel structure. Applying the same XOR to R_k recovers L_{k-1}, and the previous right half is available as L_k. This connects the two implemented steps to the way DES reverses a round.

The random mode provides inputs for a quick demonstration. To repeat that calculation, I can enter the two displayed hexadecimal values in manual mode. Showing the binary and hexadecimal forms together makes it easier to follow the bit operations and read the final 32-bit result.

Bibliography

[1] Technical University of Moldova. *Lucrare de laborator nr. 4. Cifruri bloc. Algoritmul DES*. Course assignment, p. 1, Task 2.8.

[2] National Institute of Standards and Technology. *Data Encryption Standard (DES)*. FIPS PUB 46-3, 25 October 1999, pp. 8–14 and Appendix 1. `https://csrc.nist.gov/pubs/fips/46-3/final`.